3

CPU

Futex

3.1

StarryOS `axtask` `axsched`
Future Waker

moss

`waitid`
CPU moss
Linux PollSet
Futex

3.2

3.2.1

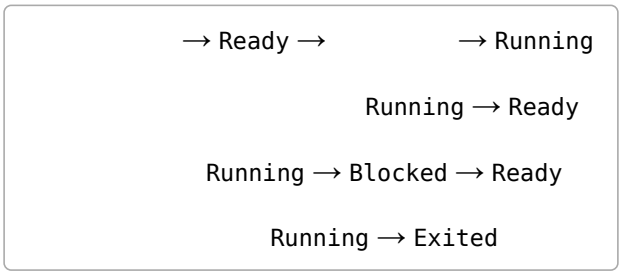
- `Running` CPU
- `Ready`
- `Blocked`
- `Exited`

```
pub enum TaskState {
    Running = 1, //      CPU
    Ready   = 2, //
    Blocked = 3, //
    Exited  = 4, //
}

pub struct TaskInner {
    //
    state: AtomicU8,
    //         1                    CPU
    cpumask: SpinNoIrq<AxCpuMask>,
    //                         CPU
    cpu_id: AtomicU32,
    // ...
}
```

3-1 CPU

`state` `cpumask` CPU `cpu_id`
“ ”



3-2

Ready Running CPU

Ready Blocked

CPU CPU

CPU CPU 0 CPU 2 CPU

CPU

CPU

```rust
fn select_run_queue_index(cpumask: AxCpuMask) -> usize {
    loop {
        //        CPU
        let index = RUN_QUEUE_INDEX.fetch_add(1, Ordering::SeqCst)
            % axconfig::plat::CPU_NUM;
        //                        CPU
        if cpumask.get(index) {
            return index;
        }
    }
}
```

3-3 CPU

CPU cpumask.get(index) CPU

3.2.2

RR

5 CPU

scheduler_timer_tick



CPU

3.2.3

CPU

	Running → Ready	
yield_now()	Running → Ready	CPU
	Running → Blocked	

3-1

Blocked resched Ready

idle

```
fn resched(&mut self) {
    //                                  idle
    let next = self.scheduler.lock().pick_next_task()
        .unwrap_or_else(|| unsafe {
            IDLE_TASK.current_ref_raw().get_unchecked().clone()
        });

    self.switch_to(crate::current(), next);
}

fn switch_to(&mut self, prev: CurrentTask, next: AxTaskRef) {
    //
    next.set_state(TaskState::Running);

    //
    let prev_ctx = prev.ctx_mut_ptr();
    let next_ctx = next.ctx_mut_ptr();
    unsafe { (*prev_ctx).switch_to(&*next_ctx) };
}
```

3-4

“ ” TaskContext

switch_to CPU

TaskContext

moss TaskContext

UserContext

→ → → →

3.2.4 CPU

CPU StarryOS 3.2.1

moss

CPU 1

Linux `sched_getaffinity` `sched_setaffinity`

```
//                          CPU
for cpu_id in 0..axconfig::plat::CPU_NUM {
    if user_mask[cpu_id / 8] & (1 << (cpu_id % 8)) != 0 {
        cpu_mask.set(cpu_id, true);
    }
}

//
let curr = current().clone();
curr.set_cpumask(cpu_mask);

//     CPU                                  CPU
if !cpu_mask.get(axhal::percpu::this_cpu_id()) {
    let migration_task = TaskInner::new(
        move || migrate_entry(curr),
        "migration-task".into(),
        4096,
    ).into_arc();
    current_run_queue().migrate_current(migration_task);
}
```

3-5 CPU

CPU CPU

CPU

Linux CPU

CPU moss StarryOS `sched_yield`

3.3

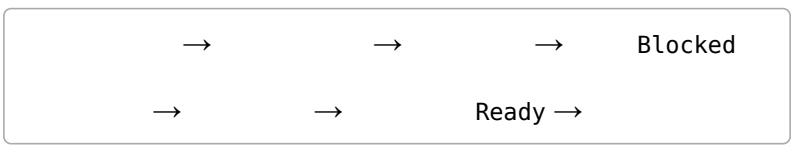
3.3.1

moss “ ”

`Blocked`

`Blocked` `Ready`

CPU



3-6

3.3.2 Waker PollSet

Waker

Waker `wake()` Waker `Blocked` `Ready`

`PollSet` Waker eventfd timerfd pidfd

moss `has_waiters`

Waker

```rust
pub struct PollSet {
    inner: Lazy<Mutex<Inner>>,
    //              wake
    has_waiters: AtomicBool,
}

pub fn wake(&self) -> usize {
    if !self.has_waiters.load(Ordering::Acquire) {
        return 0;
    }
    //                    Waker
    let (wakers, len) = self.inner.lock().take_wakers();
    self.has_waiters.store(false, Ordering::Release);
    //
    for waker in wakers.iter().take(len) {
        unsafe { waker.assume_init_read() }.wake();
    }
    len
}
```

3-7 PollSet

`take_wakers` Waker `wake()`

PollSet

PollSet “ ”

Futex

3.3.3

`block_on` Future Future `Pending` `block_on`

`Blocked` Waker `Ready` CPU

Future

```rust
fn wake_by_ref(self: &Arc<Self>) {
    if let Some(task) = self.task.upgrade() {
        //
        *self.woke.lock() = true;
        //
        select_run_queue(&task).unblock_task(task, false);
    }
}

match future.poll(&mut context) {
    //                                Blocked
    Poll::Pending if !*woke => run_queue.blocked_resched(woke),
    //
    Poll::Pending => yield_now(),
    Poll::Ready(output) => return output,
}
```

3-8 Waker block_on

`blocked_resched` `Blocked` `unblock_task` `Ready` CPU Future Waker `woke` CPU

Waker `interruptible` Future `EINTR`

3.3.4

Waker

Waker moss PollSet Futex 3.5

3.4

3.4.1

“ ”

`clock_nanosleep` Futex

3.4.2

moss (deadline, seq) BTreeMap deadline seq alarm task

```
struct TimerKey {
    //
    deadline: Duration,
    //
    seq: u64,
}

struct AlarmRuntime {
    //
    next_seq: u64,
    // BTreeMap    TimerKey
    timers: BTreeMap<TimerKey, Entry>,
}
```

3-9

deadline seq

interval timer TimerKey Key Key

EVENT_NEW_TIMER alarm task

3.4.3

nanosleep clock_nanosleep TIMER_ABSTIME moss EINTR rem LoongArch64 yield_now() Blocked Futex timeout Future

3.4.4

TimeManager times

interval timer ITIMER_REAL SIGALRM ITIMER_VIRTUAL SIGVTALRM ITIMER_PROF SIGPROF interval

3.5 Futex

3.5.1

Futex

`FutexKey` Futex

Futex

Futex `owner_dead` moss

```
pub struct WaitQueue {
    //
    //
    queue: SpinNoIrq<
        VecDeque<(Arc<AtomicBool>, u32, Arc<PollSet>)>
    >,
}
```

3-10 Futex

`AtomicBool` `notified` `u32` bitset `PollSet`

“ wake ”

3.5.2 WAIT WAKE

`FUTEX_WAIT`

`EAGAIN`

Future `FUTEX_WAKE`

`notified` PollSet

wake

```
//
if notified.load(Ordering::Acquire) {
    return Poll::Ready(());
}

//                          Waker
poll.register(cx.waker());

//              “           ”
if notified.load(Ordering::Acquire) {
    Poll::Ready(())
} else {
    Poll::Pending
}
```

3-11 Futex Waker

`notified` Waker

Waker `notified`

moss Waker

3.5.3

`FUTEX_WAIT_BITSET` `FUTEX_WAKE_BITSET` bitset mask

Futex

`FUTEX_REQUEUE` Futex

`FUTEX_CMP_REQUEUE` `EAGAIN`

`notified` bitset PollSet Waker

pthread

3.5.4

Futex moss

`interruptible(timeout(...))` `ETIMEDOUT` `EINTR`

`notified`

`FUTEX_WAIT` `FUTEX_WAIT_BITSET` moss Linux

`FUTEX_CLOCK_REALTIME`

LoongArch64 `futex_wait` pthread cancellation

3.5.5 robust list

`set_robust_list` robust list Futex

moss `futex_offset` Futex `owner_dead`

`EOWNERDEAD`

`ROBUST_LIST_LIMIT`

`clear_child_tid` Futex

`pthread_join`

3.6

3.6.1

StarryOS

`waitpid` moss `ProcessData`

`SIGCONT`

3.6.2

wait

Waker `child_exit_event`

```rust
//
match check_children().transpose() {
    Some(result) => Poll::Ready(result),
    None => {
        //                      Waker
        child_exit_event.register(cx.waker());
        //
        match check_children().transpose() {
            Some(result) => Poll::Ready(result),
            None => Poll::Pending,
        }
    }
}
```

3-12

Waker

`wake()`

3.3

`interruptible` `EINTR`

3.6.3 wait

`waitpid` pid moss

`Pid` `Pgid` `Any`

`ECHILD`

`WNOHANG`	
`WUNTRACED`	
`WCONTINUED`	
`WNOWAIT`	

3-2

`wait4` `waitid` `siginfo_t`

`CLD_EXITED` `CLD_KILLED` `CLD_STOPPED` `CLD_CONTINUED`

3.6.4 waitid pidfd

moss `waitid` `P_ALL` `P_PID` `P_PGID` `P_PIDFD` PID

`P_PIDFD`

pidfd StarryOS moss `/proc` fdinfo PID

pidfd pidfd `EAGAIN`

pidfd

`WNOWAIT`

3.7

StarryOS RR moss

PollSet

`TimerKey`

moss Futex robust list

`waitid` pidfd